

10.3.6 语音交互、大模型与项目验收

第六次课讲义 · 代码均来自本仓库 `mecamind_tools`

建议时长 2~2.5 小时：原理与架构 30~40 分钟 · 跟做 70~90 分钟 · 验收与作品集 20~30 分钟

0. 本节课要带走什么

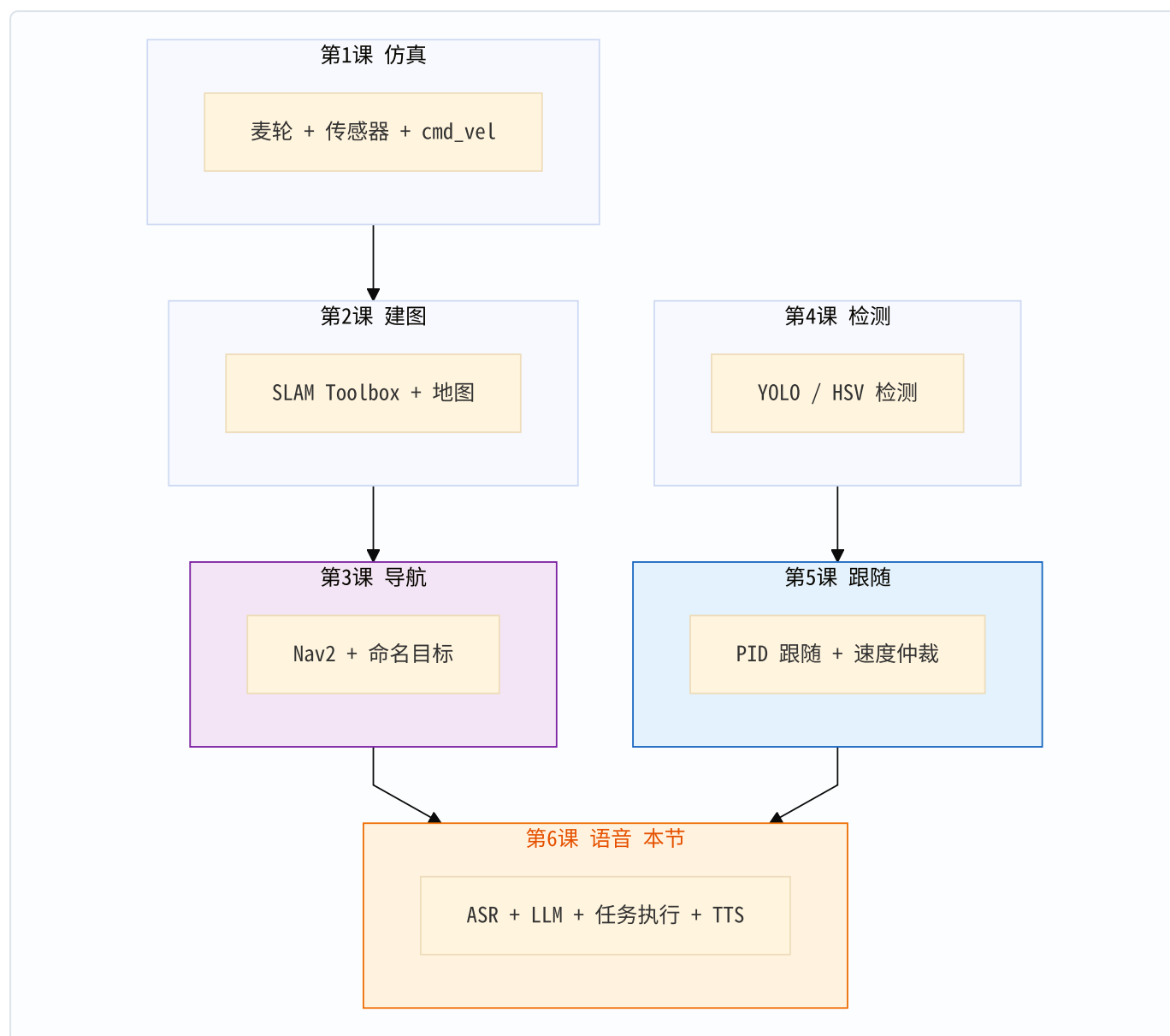
前五课分别打通了仿真底盘、建图、导航、检测、视觉跟随。本节把它们接到**语音交互层**：

1. 听懂你说的话（麦克风 / 流式 ASR / 文本兜底）
2. 理解你要做什么（规则解析或阿里云 LLM，失败自动降级）
3. **真正让车动起来**（`mission_executor` 按 intent 调 Nav2、开跟随、急停）
4. 用嘴回话（TTS + 本机播放），并完成项目级验收

一句话主线：

语音文本 → TaskCommand JSON → MissionExecutor 分发动作 → 底盘 / Nav2 / 跟随

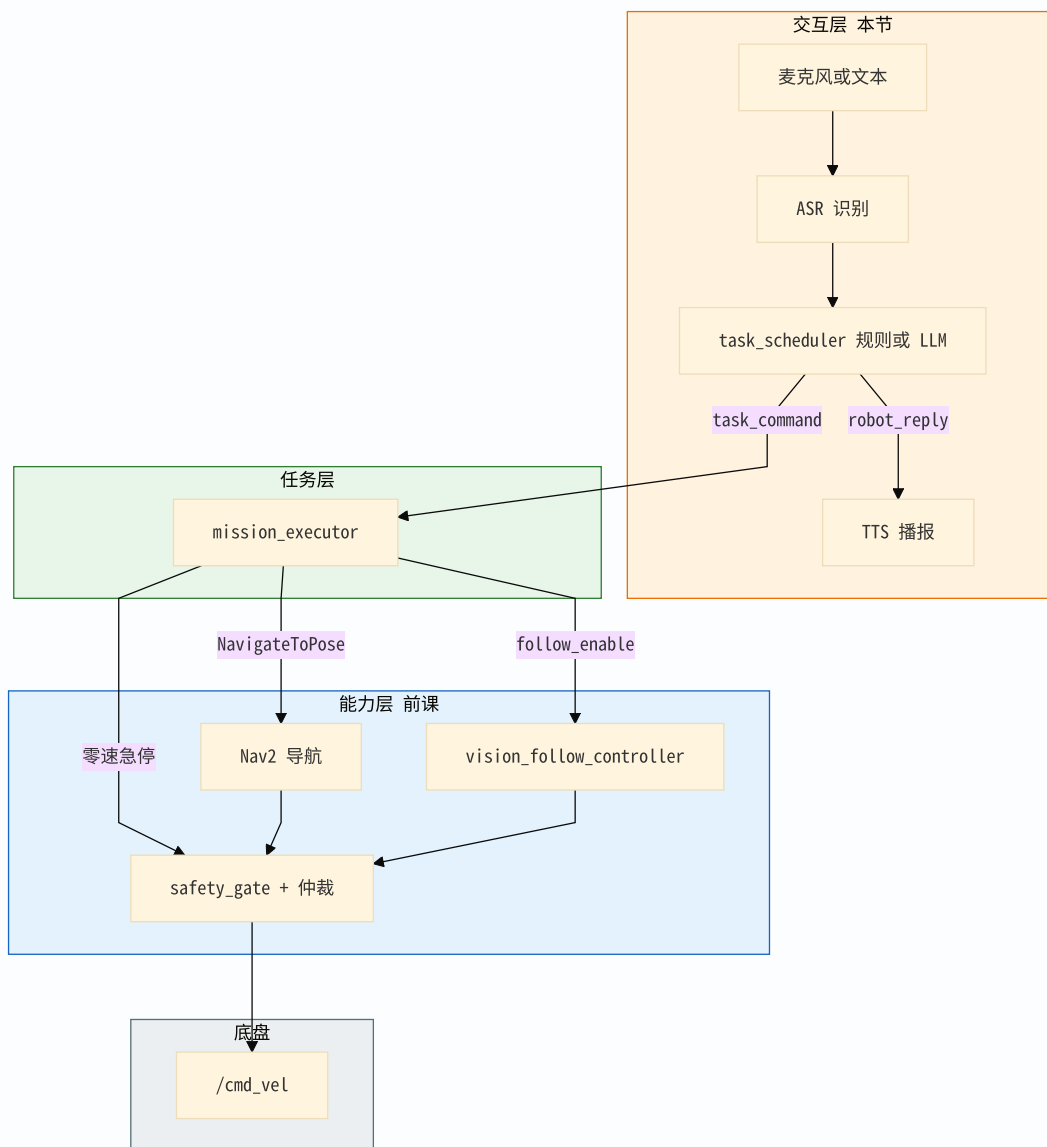
1. 六课总图与本节落点



本节在系统里的角色是**交互与任务编排**：不替代 Nav2 / PID，只负责把自然语言变成它们能消费的结构化命令。

2. 核心架构：语音如何变成动作

2.1 五层视角



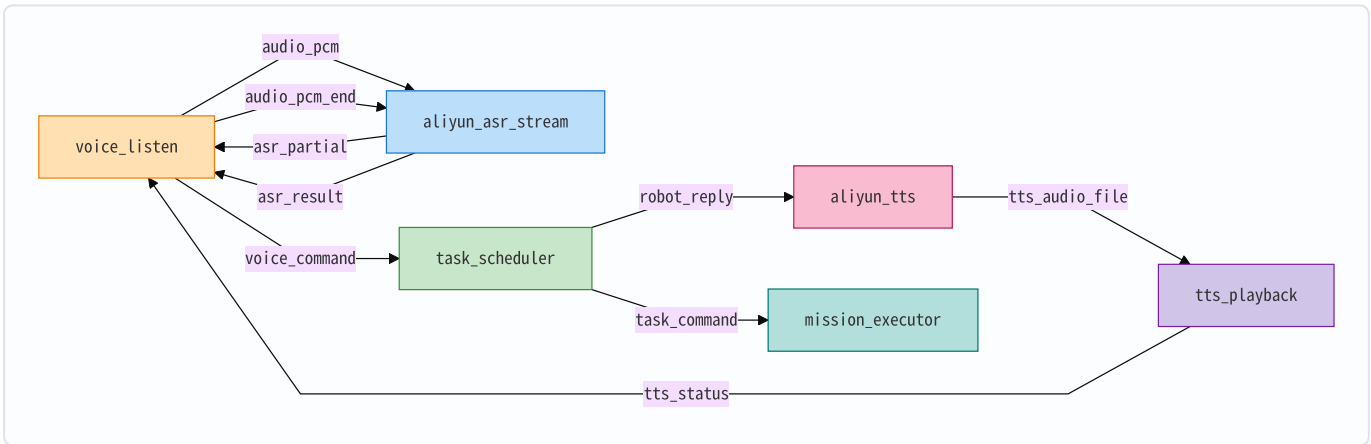
2.2 必须背下来的三段式

阶段	话题	载荷	谁生产	谁消费
听懂	/mecamind/voice_command	纯文本，如 去卧室	voice_listen / ASR / 人工 pub	task_scheduler
理解	/mecamind/task_command	JSON: intent / target / requires_confirmation / reply	task_scheduler	mission_executor
执行	Nav2 Action / follow_enable / 零 Twist	目标位姿或开关	mission_executor	Nav2 / 跟随 / 速度 链路
回话	/mecamind/robot_reply → TTS	口语字符串	task_scheduler	aliyun_tts → 播 放

LLM 不直接开车。大模型只负责把口语变成 JSON；真正动轮子的永远是 mission_executor。

2.3 默认连续听节点图

启动入口：ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py



源码锚点：

- Launch: src/mecamind_tools/launch/mecamind_aliyun_voice.launch.py
- 听麦: voice_listen_node.py → VoiceListenNode
- 流式 ASR: aliyun_speech_nodes.py → AliyunAsrStreamNode
- 客户端: aliyun_clients.py → AliyunStreamingRecognition / AliyunLlmClient
- 意图: task_scheduler.py → TaskSchedulerNode / parse_task_command
- 执行: mission_executor.py → MissionExecutorNode
- 综合接导航+跟随: mecamind_interaction.launch.py (含 scheduler + executor)

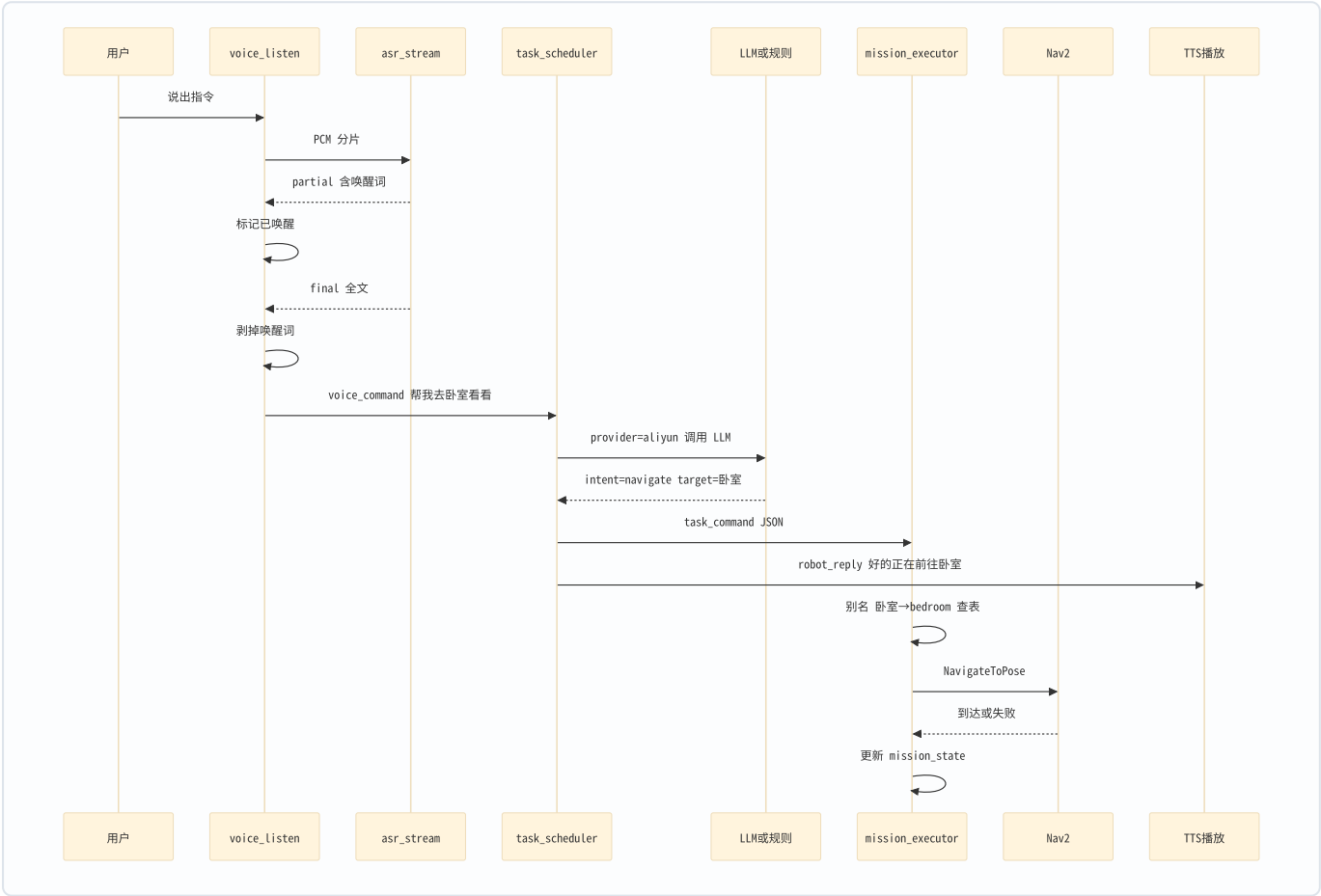
注意：纯语音 launch 默认不起 mission_executor。要让车动起来，还需导航/交互栈（见第 7 节 LAB），或自行 ros2 run mecamind_tools mecamind_mission_executor ...。课堂演示常用：先起导航，再起语

音；或文本直接灌 voice_command 验证解析。

3. 端到端主路径：语音 → LLM → 执行动作

下面以一句真实指令「小度小度，帮我去卧室看看」走完全程。这是本节最重要的一张图。

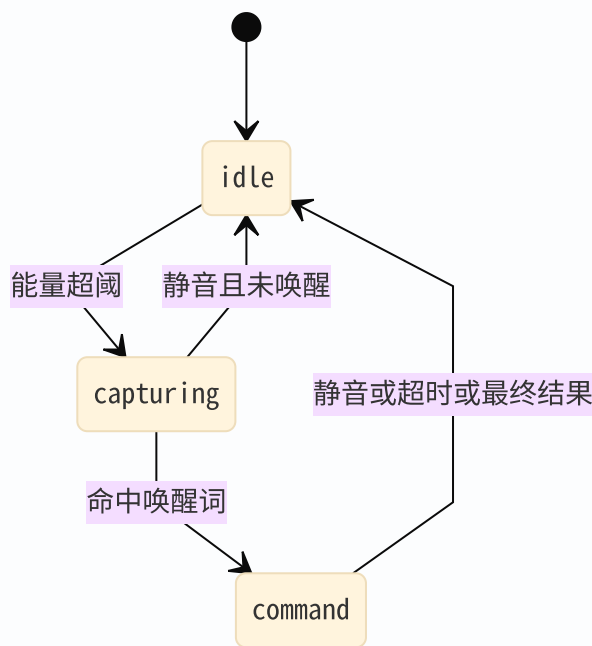
3.1 总时序



3.2 第一步：从声波到 voice_command

文件： voice_listen_node.py

状态机：



要点：

1. **能量门限**： `pcm_rms` 算 PCM 能量；启动几秒可自动标定，避免 WSL 底噪误触发。
2. **唤醒词**：默认 小度小度,小度,小杜,小渡,小肚，子串匹配，兼容 ASR 标点误转写。
3. **剥词再发令**： `strip_wake_words` 之后才发 `/mecamind/voice_command`。流式 ASR 节点设置了 `publish_voice_command:=false`，避免半句话就进调度器。
4. **TTS 互斥**：订阅 `/mecamind/tts_status`，播放中可暂停采麦，减轻「自己听自己」与 WSLg 采集停滞。

无麦克风时跳过听麦，直接：

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '去卧室'}"
```

效果与 ASR 输出等价——**调度器以下链路完全一样**。这是课堂最稳的调试入口。

3.3 第二步：从文本到 TaskCommand（规则或 LLM）

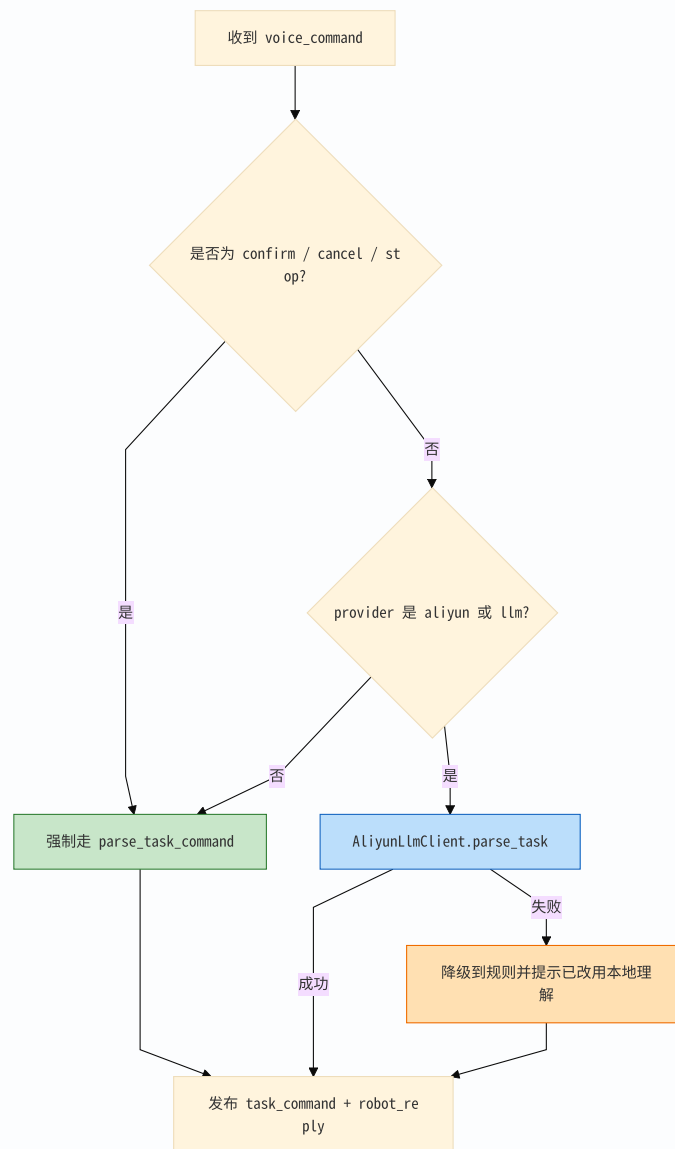
文件： `task_scheduler.py`、 `aliyun_clients.py`

统一数据结构

```
{
  "intent": "navigate",
  "target": "卧室",
  "requires_confirmation": false,
  "reply": "好的，正在前往卧室。"
}
```

intent 白名单： `stop` / `cancel` / `confirm` / `patrol` / `follow` / `mapping` / `navigate` / `unknown`。

解析决策流



为什么确认/停止永远走规则？

口头确认闭环对延迟极敏感；若「确认」还要等 LLM，用户会感觉卡死，且可能被模型乱解析。源码在 `_parse_command` 里先跑 `parse_task_command`，命中 `confirm / cancel / stop` 直接返回。

规则解析在做什么

`parse_task_command` 的优先级：

1. 停 / 急停 → `stop`
2. 取消 → `cancel`
3. 短句确认语 → `confirm`
4. 巡逻 / 跟随 / 建图 → 对应 `intent`
5. 正则 去|导航到|go to + 目标名 → `navigate`
6. 其余 → `unknown` 且 `requires_confirmation=true`

适合课堂明确口令：去卧室、开始跟随、停止。

LLM 解析在做什么

AliyunLlmClient.parse_task:

1. 用 `build_task_parser_messages` 构造 system 提示：只许输出固定 schema 的 JSON
2. POST 到 DashScope OpenAI 兼容接口 `.../chat/completions`，模型默认 `qwen-plus`，`temperature=0`
3. `parse_task_plan_payload` 剥掉 markdown 围栏、校验 intent 白名单
4. 转成 `TaskCommand` 发布

口语如「帮我去卧室看看」规则可能吃力，LLM 更容易得到 `navigate` + 卧室。

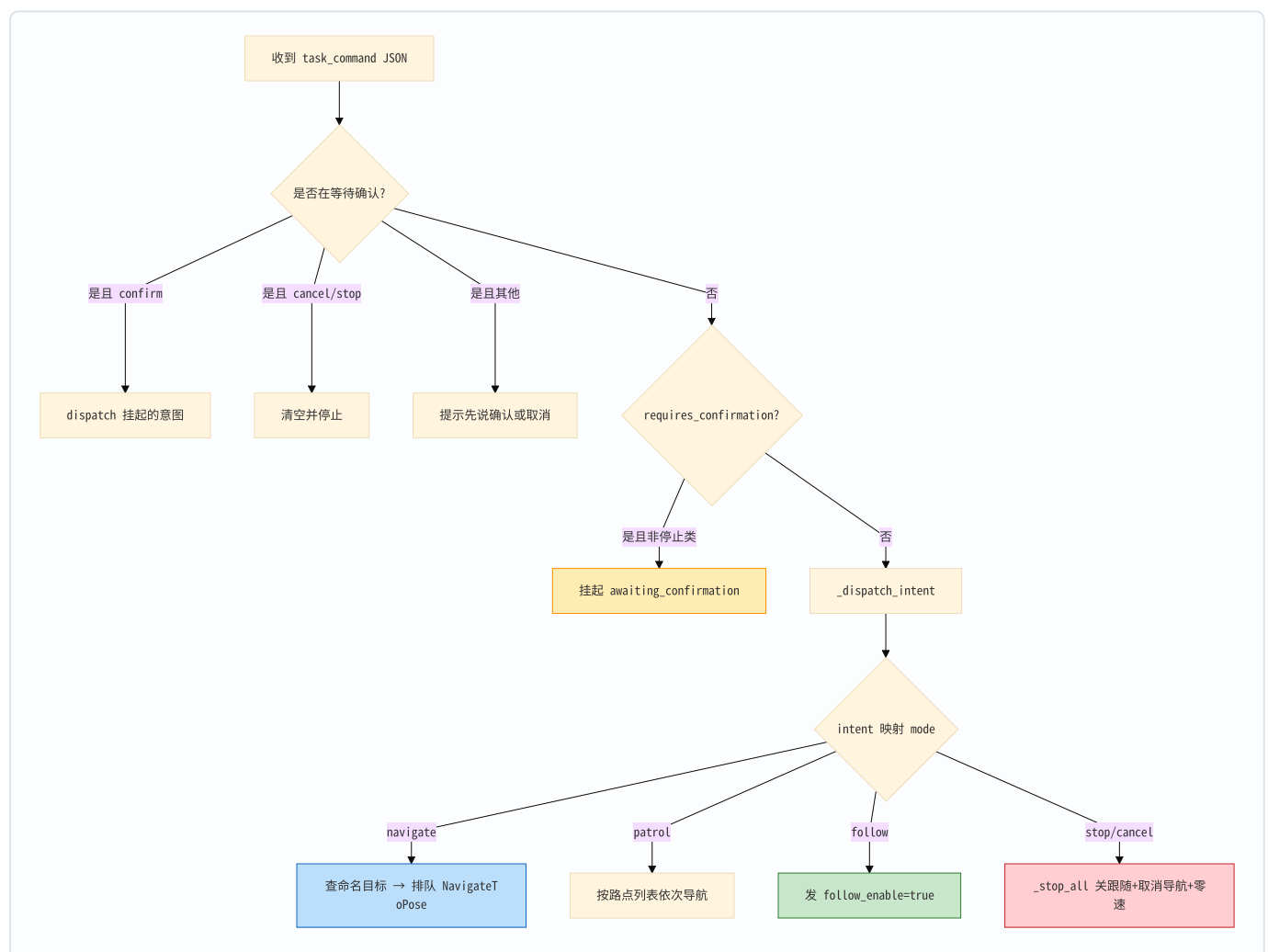
再次强调： LLM 输出的是计划，不是轮速。没有 `mission_executor`，JSON 发了车也不会动。

3.4 第三步：TaskCommand 如何变成动作

文件： `mission_executor.py`

订阅： `/mecamind/task_command`

执行入口状态机



intent → 真实动作对照表

intent	next_mission_mode	具体动作	关键函数
navigate	navigate	中英文目标别名查表 → 排队发 NavigateToPose	_enter_navigate → _send_pending
patrol	patrol	读巡航路点 YAML，逐个导航	_enter_patrol → _goal_finished 推进
follow	follow	取消导航， /mecamind/follow_enable = true	_enter_follow
stop / cancel	idle	关跟随、cancel goal、发零 Twist	_stop_all
confirm	—	执行挂起任务	_task_cb 确认分支
unknown 且需 确认	—	只挂起，不派动作	_pending_confirm
mapping	idle	当前执行器不建图，等同不支持	回到 idle

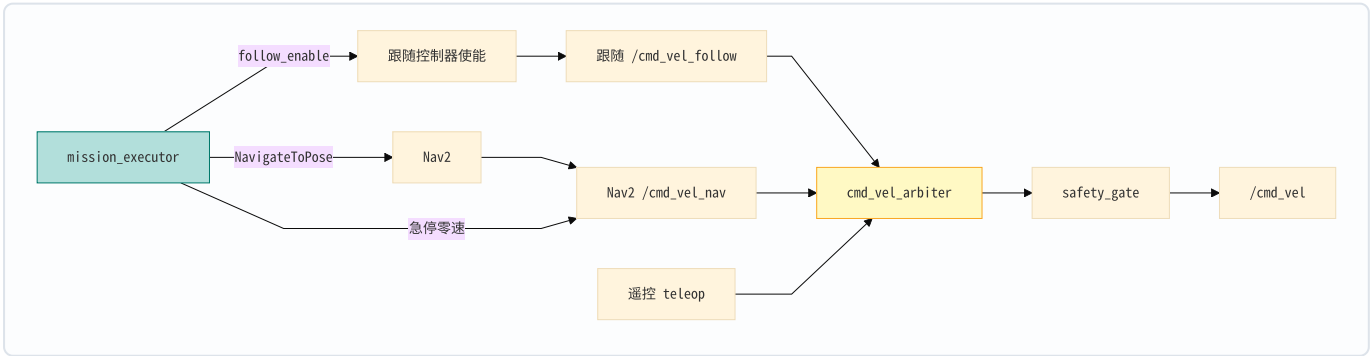
命名目标来自 config/mecamind_named_goals.yaml，中文别名在 resolve_named_goal：

口语	配置键	含义
客厅	living_room	客厅点
卧室	bedroom	卧室点
厨房	kitchen	厨房点
大厅 / 门口	hall_entry	入口点

两阶段发送为什么重要

收到 navigate 时**不立刻**调 Action，而是写入 _pending_send，由 0.5s 定时器 _tick 在 Nav2 action server 就绪后再 _send_pending。这样 Nav2 还在启动时指令不会丢，状态会显示等待 Nav2。

与第五课速度链路的关系



mission_executor 通过**开关与 Action**间接控车，不与 PID 抢写同一话题；急停时往配置的 cmd_vel_topic 发一帧零速并取消导航，保证立刻停。

3.5 第四步：回话闭环

`task_scheduler` 在发 `task_command` 的同时发 `robot_reply` → `AliyunTtsNode` 合成 wav → `TtsPlaybackNode` 用 `ffplay` / `paplay` / `aplay` 播放 → 状态回传 `voice_listen`。

模糊指令示例：

1. 用户：随便做点什么
2. 规则 → `unknown` + `requires_confirmation=true` + `reply` 「请确认是否执行…」
3. `executor` 进入 `awaiting_confirmation`，不派动作
4. 用户：确认 → 执行挂起意图；或 取消 → 清空

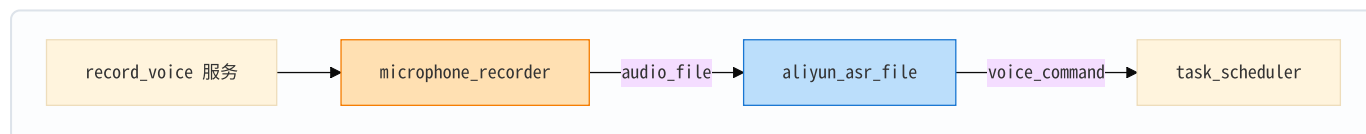
这正是 `scripts/test_voice_core.sh` 自动验收的路径。

4. 两种听模式与 PTT 兜底

4.1 continuous（默认，产品形态）

`voice_listen` + `aliyun_asr_stream`：能量触发 → 唤醒 → 最终文本发 `voice_command`。

4.2 ptt（按键说话）



```
ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py listen_mode:=ptt
ros2 service call /mecamind/record_voice std_srvs/srv/Trigger
```

录音后端 `auto`：WSL 有 `PULSE_SERVER` 时优先 `ffmpeg_pulse`，否则 `arecord`。实现见 `select_microphone_backend`。

5. 密钥、依赖与安全约定

```
cp src/mecamind_tools/config/mecamind_aliyun.example.yaml \
  src/mecamind_tools/config/mecamind_aliyun.local.yaml
# 编辑 local: aliyun.api_key: sk-...
# 或: export DASHSCOPE_API_KEY=sk-...
```

- `*.local.yaml` 已 `gitignore`，禁止提交真实密钥
- 查找顺序： `DASHSCOPE_API_KEY` 环境变量 → `load_api_key_from_config` 候选路径
- 云端 ASR/TTS： `pip install --user --break-system-packages -U dashscope`

- 课堂无网：llm_provider:=rule，文本灌 voice_command，仍可验收确认闭环

默认模型参数（launch）：

能力	参数	默认
ASR	asr_model	paraformer-realtime-v2
LLM	llm_model	qwen-plus
TTS	tts_model / tts_voice	cosyvoice-v3-flash / longanyang

6. 关键模块实现要点

6.1 voice_listen_node.py

函数 / 行为	作用
pcm_rms	16-bit PCM 能量
text_contains_wake_word / strip_wake_words	唤醒检测与剥词
_partial_cb	中间结果命中唤醒；不插播「我在」，以免掐断后半句
_result_cb	最终结果发 voice_command
TTS status 订阅	播放中可暂停采集

6.2 aliyun_clients.py

类	作用
AliyunLlmClient	HTTPS 调通义，解析任务 JSON
AliyunSpeechClient	文件 ASR / TTS
AliyunStreamingRecognition	流式 ASR WebSocket

6.3 aliyun_speech_nodes.py

节点	入口 executable
MicrophoneRecorderNode	mecamind_microphone_recorder
AliyunAsrFileNode	mecamind_aliyun_asr_file
AliyunAsrStreamNode	mecamind_aliyun_asr_stream
AliyunTtsNode	mecamind_aliyun_tts
TtsPlaybackNode	mecamind_tts_playback

另有 `classify_voice_failure`：把 ASR 空结果等异常映射成用户可读回复。

6.4 task_scheduler.py

符号	作用
<code>TaskCommand</code>	全系统统一任务结构
<code>parse_task_command</code>	零依赖规则 NLU
<code>task_command_from_aliyun_plan</code>	LLM 结果隔离转换
<code>TaskSchedulerNode._parse_command</code>	云端优先、本地兜底

6.5 mission_executor.py

符号	作用
<code>resolve_named_goal</code>	英文键 + 中文别名
<code>_task_cb</code>	确认闭环 + 分发
<code>_dispatch_intent</code>	intent → 模式入口
<code>_send_pending</code>	真正发 Nav2 goal
<code>_goal_serial</code>	作废过期异步回调

6.6 project_state_machine.py

宏观项目模式（B00T / 建图 / 导航 / ESTOP 锁存），与语音 intent 状态机**分层**：前者管「整机阶段」，后者管「当前任务」。讲义不要求当堂改代码，但验收与简历可写「急停锁存优先级」。

7. 课堂实验 LAB

LAB-0 构建与单元测试

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
colcon build --packages-select mecamind_tools --symlink-install
source install/setup.bash

python3 -m pytest src/mecamind_tools/test/test_mecamind.py \
  -k "task_scheduler or wake or voice or aliyun_llm" -q
```

预期：相关用例通过（唤醒剥词、确认解析、LLM payload、失败分类等）。

LAB-1 语音核心验收（必做，可不接仿真）

```
bash scripts/cleanup_ros2.sh
bash scripts/test_voice_core.sh
```

成功标志：MECAMIND_VOICE_CORE_OK

脚本做了什么：

1. 探测 API Key 可读（不打印全文）
2. 本地断言 确认 / 取消 解析
3. 以 `listen_mode:=ptt`、`enable_microphone:=false`、`llm_provider:=rule`、关 TTS 启动语音栈
4. 向 `voice_command` 发模糊句 → 检查 `requires_confirmation` 与回复含「确认」
5. 再发「确认」→ 检查 `intent == confirm`

LAB-2 文本路径手调调度器

```
ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py \
  listen_mode:=ptt enable_microphone:=false llm_provider:=rule
```

另开终端：

```
ros2 topic echo /mecamind/task_command
ros2 topic echo /mecamind/robot_reply

ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '去卧室'}"
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '开始跟随'}"
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '停止'}"
```

观察 JSON 的 `intent` / `target` / `reply` 是否符合预期。

可选：把 `llm_provider:=aliyun`，再说「帮我去卧室看看」，对比规则与 LLM 的 `target` / `reply`。

LAB-3 连续听 + TTS（有麦）

```
ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py
# 说：小度小度去卧室
# 听：TTS 回复；echo task_command
```

无唤醒调试可临时改 `wake_words:=` 为空，或继续用文本 pub。

LAB-4 真正让车动起来（选做 / 综合演示）

需要：已有地图、Nav2 可导航；跟随链路按第五课。

推荐顺序：

1. 按第三节启动导航（或项目 bringup 导航入口）

2. 启动交互层（含 scheduler + executor）：

```
ros2 launch mecamind_tools mecamind_interaction.launch.py \  
  task_provider:=rule enable_fake_detections:=true
```

3. 再开语音，或直接文本：

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '去卧室'}"  
ros2 topic echo /mecamind/mission_state
```

预期：mission_state 出现 mode=navigate、detail 含 queued:bedroom / reached:...；RViz 出现导航目标。
跟随：

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '开始跟随'}"  
# mission_state: mode=follow；跟随控制器使能  
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '停止'}"
```

历史综合入口 mecamind_demo_ui.launch.py 可一并拉导航+交互；开麦时 enable_microphone:=true，且该入口里语音 launch 关闭了内置 scheduler（避免与 interaction 双重调度），以 interaction 侧为准。

8. 验收清单与故障排查

8.1 本节验收

项	命令 / 现象	通过标准
单元	pytest 语音相关	绿
核心 smoke	test_voice_core.sh	MECAMIND_VOICE_CORE_OK
文本→任务	pub 去卧室	task_command 含 navigate + 卧室类 target
确认闭环	模糊句 → 确认	先挂起再 confirm
云端可选	provider=aliyun	口语可解析；失败有规则降级日志
综合可选	接 Nav2	mission_state 反映到达；停止立刻 idle

8.2 常见故障

现象	可能原因	处理
smoke 报缺 Key	未配 local / 环境变量	复制 example 或 export
有 JSON 车不动	只起了语音栈	起导航 + mission_executor
unknown_goal	目标名不在别名表	用客厅/卧室/厨房或英文键
唤醒不灵	能量门限过高 / 无麦	调低 energy_threshold ; 改用文本
TTS 时麦卡死	WSLg Pulse 停滞	依赖节点看门狗; 或 PTT
跟随时还在导航	模式未互斥清理失败	先说停止; 查 mission_state
LLM 慢或失败	网络 / Key	自动降级 rule; 确认短指令仍走规则

清理残留：

```
bash scripts/cleanup_ros2.sh
```

9. 项目收尾：演示脚本与简历表述

9.1 两分钟演示脚本建议

- 1. 文本路径：模糊指令 → TTS/回复「请确认」→「确认」→ 状态变化
- 2. 明确指令：去厨房 → RViz 路径 → 到达
- 3. 开始跟随 → 红柱/目标移动 → 停止
- 4. 可选：唤醒词真实说话一条

9.2 简历可写要点（对齐真实代码）

- 基于 ROS 2 的语音任务链路：流式 ASR、规则/LLM 双 Provider、结构化 TaskCommand
- LLM 失败自动降级；确认/急停短指令强制本地解析
- mission_executor 将 intent 映射到 Nav2 Action、跟随使能与急停，与速度仲裁解耦
- 无麦文本路径与 test_voice_core.sh 保证课堂/CI 可验收

10. 源码阅读顺序（课后）

- 1. task_scheduler.py : parse_task_command → _parse_command
- 2. aliyun_clients.py : AliyunLlmClient.parse_task → parse_task_plan_payload
- 3. mission_executor.py : _task_cb → _dispatch_intent → _enter_navigate

- 4. `voice_listen_node.py`：唤醒状态机与 `_result_cb`
- 5. `mecamind_aliyun_voice.launch.py`：节点如何拼成一条链

附录 A 话题速查

名称	类型	说明
<code>/mecamind/audio_pcm</code>	String	基64 PCM 块
<code>/mecamind/audio_pcm_end</code>	String	结束流式会话
<code>/mecamind/asr_partial</code>	String	中间识别
<code>/mecamind/asr_result</code>	String	最终识别
<code>/mecamind/voice_command</code>	String	自然语言指令
<code>/mecamind/task_command</code>	String JSON	结构化任务
<code>/mecamind/robot_reply</code>	String	TTS 文案
<code>/mecamind/tts_audio_file</code>	String	wav 路径
<code>/mecamind/tts_status</code>	String	播放状态
<code>/mecamind/mission_state</code>	String JSON	执行状态
<code>/mecamind/follow_enable</code>	Bool	跟随开关
<code>/mecamind/record_voice</code>	Trigger 服务	PTT 录音

附录 B 与规划文档的对应

`PROJECT_PLAN.md` 10.3.6：麦克风 / ASR / LLM / TTS、导航·巡逻·跟随·停止、云端失败兜底、一键验收。
本讲义以仓库已实现的 `mecamind_aliyun_voice` + `task_scheduler` + `mission_executor` + `test_voice_core.sh` 为准；全仓 `test_cp6` 若后续补齐，验收命令以 `README` 更新为准。